

JOUR

04

sur 05

DATE DE SESSION

*lundi 8 juin 2026*

Durée : 4h en présentiel

# Mémoire & passage au C++

## Programme de la journée

Anatomie mémoire d'un programme : stack et heap

Allocation dynamique : malloc, free, fuites mémoire

Du C au C++ : iostream, classes, encapsulation

Atelier — gestion d'un répertoire d'étudiants

# Aujourd'hui : la mémoire et le passage au C++

*Deux gros morceaux à digérer*

La gestion mémoire est le sujet le plus délicat du C — et c'est aussi ce qui en fait sa puissance. Vous allez comprendre où vivent vos variables, comment en allouer plus à la demande, et pourquoi un programmeur C peut produire de magnifiques bugs.

Ensuite, on franchit le pont vers C++ : un langage qui ajoute la programmation orientée objet, une bibliothèque standard plus riche, et des constructions modernes — tout en restant compatible avec le C.

## MATIN — MÉMOIRE

Stack vs heap, allocation statique vs dynamique, malloc/free, calloc/realloc, fuites mémoire, segfault, double free.

## APRÈS-MIDI — C++

Différences C / C++, iostream, classes, attributs et méthodes, encapsulation, constructeur, destructeur.

# Anatomie mémoire d'un programme C

4 zones, chacune avec ses règles

## Stack (pile)

Variables locales, paramètres de fonction. Croît vers le bas. Petite (~8 Mo). Gérée automatiquement.

Adresses hautes

## Heap (tas)

Allocation dynamique avec malloc. Croît vers le haut. Grand (limité par la RAM). Gérée par le programmeur.

## BSS / Données

Variables globales et statiques. Persiste pendant toute l'exécution.

## Code (text)

Le code machine du programme. Lecture seule.

Adresses basses

### À RETENIR

Stack et heap sont les deux zones que vous manipulerez le plus.

Stack = automatique, rapide, mais petite et limitée à la durée de vie d'une fonction.

Heap = manuel (malloc/free), plus lent, mais grand et persistant tant que vous ne libérez pas.

Le choix entre les deux est un choix d'architecture.

# Stack vs Heap — deux mondes

*Quand utiliser l'un ou l'autre ?*



## Stack (la pile)

- Allocation et libération automatiques
- Très rapide (juste un déplacement de pointeur)
- Variables locales et paramètres de fonction
- Petite (~8 Mo par défaut)
- Durée de vie limitée à la fonction qui l'a créée
- Pas de risque de fuite mémoire
- Risque de "stack overflow" si récursion infinie



## Heap (le tas)

- Allocation manuelle (malloc / new) et libération manuelle (free / delete)
- Plus lent (gestion complexe)
- Limitée par la RAM disponible (plusieurs Go)
- Persiste tant qu'on ne libère pas
- Permet de retourner des structures depuis une fonction
- Risque de fuite mémoire si on oublie free
- Risque de double free, dangling pointers, etc.

# Allocation statique (sur la pile)

La méthode classique vue jusqu'ici

```
main.c — gcc main.c -o main

#include <stdio.h>

void afficher(void) {
    int  tableau[1000];      // 1000 ints sur la PILE
    int  total = 0;
    for (int i = 0; i < 1000; i++) {
        tableau[i] = i * 2;
        total += tableau[i];
    }
    printf("Total : %d\n", total);
    // tableau est détruit AUTOMATIQUEMENT en sortie de fonction
}

int main(void) {
    afficher();
    afficher();
    return 0;
}
```

- `tableau[1000]` est alloué sur la pile : créé à l'entrée de `afficher()`, détruit à la sortie.
- Aucun `free` à écrire — c'est la magie (et la limite) de la pile.
- Mais : impossible de retourner ce tableau depuis `afficher()` — il sera détruit avant que l'appelant l'utilise. C'est là qu'on a besoin du tas.

# Allocation dynamique : malloc et free

*Demander de la mémoire au système — et la rendre*

```
main.c — gcc main.c -o main

#include <stdio.h>
#include <stdlib.h>           // pour malloc et free

int main(void) {
    int n;
    printf("Combien de notes ? ");
    scanf("%d", &n);

    // Reserver n * sizeof(int) octets sur le tas
    int *notes = (int *) malloc(n * sizeof(int));
    if (notes == NULL) {
        printf("Echec d'allocation\n");
        return 1;
    }
    // Remplir et utiliser
    for (int i = 0; i < n; i++) notes[i] = 10 + i;

    // ... travail ...

    free(notes);           // RENDRE la memoire au systeme
    notes = NULL;          // eviter de reutiliser un pointeur libere
    return 0;
}
```

- malloc(t) réserve t octets et retourne un pointeur sur le début. NULL si l'allocation a échoué — toujours vérifier !
- free(p) libère la mémoire pointée par p. Sans free → fuite mémoire (la mémoire reste réservée jusqu'à la fin du programme).
- Mettre le pointeur à NULL après free évite les usages accidentels ("dangling pointers").

# Variantes : calloc, realloc

## Initialisation à zéro et redimensionnement

```
main.c - gcc main.c -o main

#include <stdio.h>
#include <stdlib.h>

int main(void) {
    // calloc : alloue ET initialise tous les octets à 0
    int *t = (int *) calloc(10, sizeof(int));
    // t[0]..t[9] valent tous 0 (alors qu'avec malloc, contenu indefini)

    // Besoin de plus de place ? realloc agrandit le bloc
    t = (int *) realloc(t, 20 * sizeof(int));
    // Les 10 premières valeurs sont preservatives
    // Les 10 nouvelles cases sont indefinies

    // free fonctionne pareil
    free(t);

    return 0;
}
```

- `calloc(n, t) = malloc(n*t) + initialisation à 0`. Plus lent, mais plus sûr pour les zones critiques.
- `realloc(p, t)` tente d'agrandir le bloc. Si pas possible, copie les données dans un nouveau bloc et libère l'ancien.
- Toujours réassigner le résultat de `realloc` — l'adresse peut changer !

# Les bugs mémoire classiques

*Connaître les ennemis pour mieux les éviter*

## Fuite mémoire

Vous allouez avec malloc, mais oubliez de free. La mémoire reste réservée jusqu'à la fin du programme. Sur un serveur 24/7 : crash garanti.

## Segfault (segmentation fault)

Vous accédez à une adresse interdite : `*p` quand `p == NULL`, dépassement de tableau. Le système tue le programme.

## Double free

Vous appelez `free(p)` deux fois sur le même pointeur. Comportement indéfini, souvent crash immédiat.

## Use after free

Vous utilisez un pointeur après l'avoir free. La mémoire a peut-être été réutilisée — données aléatoires ou crash.

## Buffer overflow

Écrire au-delà de la taille allouée. La cause #1 des failles de sécurité historiques (Heartbleed, Morris worm...).

## Variable non initialisée

Lire le contenu d'une variable avant de l'avoir assignée. Le contenu est aléatoire — bugs intermittents difficiles à reproduire.

*Demain (Jour 5), nous verrons des outils — Valgrind notamment — qui détectent automatiquement la plupart de ces bugs.*



# Règles d'or de la mémoire en C

*Sept règles qui vous éviteront 80% des bugs*

---

**Tout malloc se termine par un free** Au moment où vous écrivez malloc, écrivez tout de suite le free correspondant. Sinon, vous oublierez.

**Toujours vérifier le retour de malloc** if (p == NULL) { gestion d'erreur; }. Allouer peut échouer.

**Mettre le pointeur à NULL après free** Ça désactive les usages accidentels et facilite la détection de bugs.

**Ne jamais free deux fois** Si p est mis à NULL après free, free(NULL) est silencieux et sans effet — protection naturelle.

**Ne jamais retourner l'adresse d'une variable locale** Elle est sur la pile et sera détruite en sortie de fonction. Utilisez malloc à la place.

**Initialiser ses variables** int x = 0; pas juste int x;. Une variable non initialisée contient du "bruit".

**Tester votre code avec Valgrind (Jour 5)** Cet outil exécute votre programme et signale toutes les anomalies mémoire.

# Du C au C++ — ce qui change

*C++ étend C, mais ajoute beaucoup d'idées*



- Procédural pur
- Pas de classes
- Allocation : malloc / free
- Entrées/sorties : printf / scanf
- Pas d'exceptions (gestion d'erreurs par codes de retour)
- Pas de templates / génériques
- Bibliothèque standard restreinte



- Multi-paradigme : procédural + OO + générique
- Classes, héritage, polymorphisme
- Allocation : new / delete (et malloc reste utilisable)
- Entrées/sorties : std::cout, std::cin (iostream)
- Exceptions (try / catch / throw)
- Templates : conteneurs et algorithmes génériques (STL)
- STL : vector, string, map, sort, etc.

# Hello World en C++

*iostream replace stdio.h*

```
main.c — gcc main.c -o main

// hello.cpp
#include <iostream>
#include <string>

int main() {
    std::string nom;
    std::cout << "Quel est ton nom ? ";
    std::cin >> nom;

    std::cout << "Bonjour " << nom << " !" << std::endl;

    int age = 19;
    std::cout << "Tu as " << age << " ans" << std::endl;

    return 0;
}
```

- Compilation : `g++ hello.cpp -o hello` (g++ au lieu de gcc).
- `std::cout << ...` — l'opérateur `<<` "envoie" la valeur vers la sortie. Pas besoin de format `%d`, `%s` : C++ détecte le type.
- `std::endl` ajoute un saut de ligne et vide le tampon. `"\n"` fait sauter une ligne sans vider — plus rapide.
- `std::string` est une vraie chaîne (avec gestion automatique de la mémoire), bien plus facile que `char[]` en C.

# Les classes en C++ — la POO

*Encapsuler données et comportements ensemble*

```
main.c — gcc main.c -o main

// point.cpp
#include <iostream>

class Point {
private:
    double x;          // attributs caches (encapsulation)
    double y;
public:
    // Constructeur : appelle a la creation d'un Point
    Point(double xi, double yi) { x = xi; y = yi; }
    // Methodes : actions sur l'objet
    double getX() const { return x; }
    double getY() const { return y; }
    void afficher() const {
        std::cout << "(" << x << ", " << y << ")\n";
    }
};

int main() {
    Point p1(3.0, 4.0);
    Point p2(10.0, 20.0);
    p1.afficher();    p2.afficher();
    return 0;
}
```

- Une classe regroupe des attributs (données) et des méthodes (fonctions qui agissent dessus).
- private: limite l'accès depuis l'extérieur — l'encapsulation cache les détails internes.
- Le constructeur a le même nom que la classe et est appelé automatiquement à la création (Point p1(3, 4)).

# Constructeur et destructeur

## Cycle de vie d'un objet

```
main.c - gcc main.c -o main

class Compteur {
private:
    int valeur;
    int *historique;    // tableau dynamique
    int taille;
public:
    Compteur(int n) {           // CONSTRUCTEUR
        valeur = 0; taille = n;
        historique = new int[n]; // alloue sur le tas
        std::cout << "Compteur cree\n";
    }
    ~Compteur() {               // DESTRUCTEUR
        delete[] historique;    // libere la memoire
        std::cout << "Compteur detruit\n";
    }
    void incrementer() { valeur++; }
    int lire() const { return valeur; }
};

int main() {
    Compteur c(100);           // constructeur appele automatiquement
    c.incrementer(); c.incrementer();
    std::cout << c.lire() << "\n";
    // Destructeur appele AUTOMATIQUEMENT a la sortie
}
```

- Constructeur : initialise les attributs, alloue les ressources (mémoire, fichiers, connexions...).
- Destructeur (~Classe) : libère les ressources. Appelé automatiquement quand l'objet sort de portée.
- C'est le pattern RAII : Resource Acquisition Is Initialization. C'est le grand atout du C++ sur le C.
- `new[]` se libère avec `delete[]`, pas `delete` (qui libère un seul élément).

# Pourquoi encapsuler ?

*Trois grandes raisons d'utiliser private/public*

**Protéger l'intégrité des données** Si l'attribut `age` est `private`, personne ne peut lui mettre -50 par accident. Toute modification passe par une méthode qui peut valider.

**Cacher la complexité interne** L'utilisateur de la classe voit l'interface (méthodes publiques), pas l'implémentation. Vous pouvez changer l'intérieur sans casser le code des autres.

**Faciliter la maintenance** Les bugs liés à un attribut sont localisés dans la classe — vous savez où chercher.

**Bonnes pratiques classiques** Tous les attributs en `private`, des getters et setters publics si nécessaire.

**Modificateurs d'accès** `private` : visible uniquement dans la classe.  
`protected` : aussi dans les classes filles. `public` : partout.

## ANALOGIE

Une voiture : vous avez accès au volant, à la pédale (interface publique).

Les bougies, les engrenages, l'allumage électronique sont cachés sous le capot (interne, privé).

Un changement d'allumage ne vous oblige pas à réapprendre à conduire.

# Mini-application — classe Etudiant

*Tout ce qu'on a vu en une seule classe*

```
main.c — gcc main.c -o main

#include <iostream>
#include <string>

class Etudiant {
    std::string nom;
    int age; double moyenne;
public:
    Etudiant(std::string n, int a) : nom(n), age(a), moyenne(0.0) {}
    void ajouterNote(double note) { moyenne = (moyenne + note) / 2.0; }
    bool estMajeur() const { return age >= 18; }
    void afficher() const {
        std::cout << nom << " (" << age << " ans)"
                    << " - moyenne: " << moyenne << "\n";
    }
};

int main() {
    Etudiant ayoub("Ayoub", 25);
    ayoub.ajouterNote(15);
    ayoub.ajouterNote(17);
    ayoub.afficher();
    if (ayoub.estMajeur()) std::cout << "Majeur\n";
    return 0;
}
```

- Un Etudiant a un nom, un âge, une moyenne — privés par défaut dans une classe (pas besoin du mot-clé private).
- Les méthodes publiques offrent les opérations possibles : ajouter une note, vérifier la majorité, afficher.
- Le constructeur utilise la liste d'initialisation (: nom(n), age(a)...) — plus efficace que l'affectation.

# Atelier 4 — Répertoire d'étudiants

## Atelier pratique



### MISSION

Construisez un programme C++ qui gère une promotion : ajouter, afficher, calculer la moyenne globale.

### Étapes à réaliser

1. Créer une classe Etudiant avec nom (string), age (int) et note (double), constructeur, getters et afficher().
2. Dans main(), créer un tableau dynamique de 5 Etudiant (utiliser new ou un std::vector si vous êtes à l'aise).
3. Saisir au clavier les infos de chaque étudiant.
4. Afficher tous les étudiants, calculer la note moyenne de la promotion.
5. Trouver et afficher l'étudiant ayant la meilleure note.



### ASTUCE ·

Si vous utilisez new, n'oubliez pas le delete[] à la fin. Pensez RAI ! : la classe doit se libérer toute seule.



# Bilan du Jour 4

*Ce que vous savez maintenant faire*



## ACQUIS DE LA JOURNÉE

- Distinguer la pile et le tas
- Allouer dynamiquement avec malloc/calloc/realloc et libérer avec free
- Identifier les bugs mémoire courants (fuite, segfault, double free)
- Compiler du C++ avec g++ et utiliser std::cout / std::cin
- Définir une classe avec attributs privés et méthodes publiques
- Utiliser un constructeur et un destructeur pour gérer les ressources



## PROCHAINE ÉTAPE

Demain c'est le dernier jour. On va boucler la boucle : pourquoi compilation et runtime sont deux mondes différents, comment optimiser un programme avec les bons drapeaux GCC, comment déboguer avec gdb et chasser les fuites avec Valgrind, et comment organiser un projet sérieux avec un Makefile.

Nous présenterons aussi le projet final qui validera votre acquis.

Bon repos — la mémoire, c'est costaud.